

Environnement SBT & Scala avec Visual Studio Code

*Installer, configurer et construire un pipeline de données fonctionnel avec Option, Either et le chaînage
map / flatMap / fold / for*

DÉROULÉ DE LA SÉANCE

1

Cours

Environnement SBT, VS Code + Metals, puis les fondamentaux fonctionnels : Option, Either, map, flatMap, fold, for.

2

Étude de cas

Les étudiants codent un pipeline de validation de commandes en autonomie, à partir d'un squelette fourni.

3

Correction

Relecture collective de la solution, points de vigilance et erreurs fréquentes.

4

Validation environnement

Chaque étudiant démontre une installation fonctionnelle : JDK, SBT, VS Code, Metals, Git.

À l'issue du cours, vous saurez...

1

Installer un environnement Scala complet

JDK, SBT et VS Code avec l'extension Metals, prêts pour le développement.

2

Créer et faire évoluer un projet SBT

Structure standard, commandes compile / run / test, et publication sur GitLab.

3

Sécuriser un pipeline avec Option et Either

Représenter l'absence de valeur et les erreurs métier de façon explicite et typée.

4

Châîner des transformations lisibles

Maîtriser map, flatMap, fold et la syntaxe for pour construire un pipeline propre.

Qu'est-ce que SBT ?

Simple Build Tool — l'outil qui compile, teste, package et automatise vos projets Scala.

1

Compiler

Transforme les fichiers .scala en bytecode Java.

2

Gérer les dépendances

Télécharge automatiquement les bibliothèques depuis Maven Central.

3

Structurer le projet

Impose une organisation standard (src/main/scala, src/test/scala).

4

Exécuter et tester

Lance l'application (sbt run) et les tests (sbt test).

5

Packager

Génère des fichiers .jar prêts à déployer.

6

Automatiser

S'intègre dans des pipelines CI/CD (GitLab CI, Jenkins, GitHub Actions).

Pourquoi utiliser SBT ?

SBT face à l'écosystème Java

- L'équivalent de Maven / Gradle pour Java, mais pensé pour Scala.
- Simplifie la gestion des projets complexes (multi-modules, dépendances).
- Reconnu comme la norme dans l'écosystème Scala.
- S'intègre nativement avec Metals dans VS Code.

Comparatif rapide



ÉTAPE 1

Installer Scala + SBT

1

Installer le JDK

Scala nécessite un JDK — version 17 recommandée (minimum JDK 11).

2

Installer SBT

Outil de build officiel, à télécharger depuis scala-sbt.org.

3

Vérifier l'installation

Contrôler les deux commandes ci-contre dans un terminal.



```
$ java -version  
openjdk version "17.0.x"  
  
$ sbt sbtVersion  
[info] 1.9.x
```

À retenir

JAVA_HOME doit pointer vers le JDK installé. C'est la cause la plus fréquente de blocage à cette étape.

ÉTAPE 2

Configurer Visual Studio Code

1

Installer VS Code

L'éditeur qui accueillera l'ensemble du projet Scala.

2

Extension Metals

Le Scala Language Server : autocomplétion, navigation, refactoring, exécution.

3

Extension Scala Syntax

Coloration syntaxique officielle (optionnelle mais recommandée).

4

Ouvrir le projet

Metals détecte automatiquement build.sbt et propose d'importer le projet.

Structure standard d'un projet SBT

```
moteur-recommandation/  
├─ build.sbt  
├─ project/  
│   └─ build.properties  
├─ src/  
│   ├── main/scala/  
│   └─ test/scala/
```

build.sbt

Configuration du projet : version de Scala, dépendances, options de build.

project/build.properties

Fixe la version de SBT utilisée par le projet.

src/main/scala/

Le code source de l'application.

src/test/scala/

Les tests unitaires du projet.

Metals détecte automatiquement cette structure et active l'environnement Scala.

ÉTAPE 3

Créer et lancer un projet Scala

1. Générer le projet depuis un template :

```
● ● ●  
  
$ sbt new scala/scala-seed.g8  
  
name [My Something Project]:  
> moteur-recommandation
```

2. Lancer le projet :

```
● ● ●  
  
$ cd moteur-recommandation  
$ sbt run
```

Exemple minimal — Hello World

```
● ● ●  
  
// build.sbt  
scalaVersion := "3.3.1"
```

```
● ● ●  
  
// src/main/scala/Hello.scala  
@main def helloWorld =  
  println("Hello, world")
```

Lancer avec : sbt run

Les commandes SBT de base

`sbt compile`

Lit build.sbt, télécharge les dépendances et compile src/main/scala en bytecode dans target/.

`sbt run`

Exécute l'application à partir des classes compilées.

`sbt test`

Compile et exécute les tests présents dans src/test/scala.

Astuce VS Code : palette de commandes (Ctrl+Shift+P) → "Metals: Run" ou "Metals: Test" pour lancer sans quitter l'éditeur.

Lire un résultat de compilation

Succès



```
[success] Total time: 3 s,  
completed Jul 28, 2026, 13:05:12
```

Les fichiers compilés sont disponibles dans target/scala-<version>/classes.

Erreur



```
[error] src/main/scala/Hello.scala:5:  
      not found: value printlnn  
[error] printlnn("Hello, world")
```

SBT indique le fichier, la ligne et la nature de l'erreur : corrigez, puis relancez sbt compile.

Étape suivante

Une fois la compilation réussie, sbt run exécute l'application en utilisant les classes compilées dans target/.

Workflow Git & GitLab

```
$ git init
$ git remote add origin \
  https://gitlab.com/user/projet.git
$ git add .
$ git commit -m "Initial commit"
$ git push -u origin master
```

1

Créer un compte GitLab puis un nouveau projet vide.

2

Copier l'URL du dépôt et la relier au projet local.

3

Committer régulièrement avec des messages explicites.

4

Travailler en équipe via des branches (git checkout -b feature-similarity).

Bonnes pratiques à adopter



README.md

Toujours expliquer comment lancer le projet.



Exécution via Metals

Lancer directement les fichiers Scala depuis VS Code.



Documentation du code

Des commentaires clairs sur chaque fonction.



Commits fréquents

Petits commits explicites plutôt que de gros commits rares.

```
Option[A] ·  
Either[E, A]
```

PARTIE 2

Programmation fonctionnelle pour pipelines de données

Option, Either, et le chaînage map · flatMap · fold · for

Option[A]

Un conteneur à deux états

- `Some(valeur)` — la valeur existe.
- `None` — la valeur est absente.
- Remplace avantageusement `null` : le type `Option[A]` force à traiter le cas d'absence.
- Idéal pour un parsing qui peut échouer sans qu'on ait besoin de savoir pourquoi.



```
def toIntOpt(s: String): Option[Int] =  
  s.toIntOption  
  
toIntOpt("12")    // Some(12)  
toIntOpt("abc")   // None
```

Dans le pipeline de l'étude de cas

Option servira à convertir en toute sécurité une quantité ou un prix saisis en texte vers un nombre.

Either[E, A]

Un résultat, deux issues possibles

- Right(valeur) — le traitement a réussi.
- Left(erreur) — le traitement a échoué, avec un message explicite.
- Là où Option dit seulement "rien", Either explique pourquoi.
- Parfait pour la validation de règles métier (quantité, prix, format).



```
def validerQuantite(q: Int)
  : Either[String, Int] =
  if (q > 0) Right(q)
  else Left("Quantité invalide")
```

Option vs Either

Option : "y a-t-il une valeur ?" — Either : "quelle valeur, ou quelle erreur précise ?"

map : transformer une valeur

Applique une fonction à l'intérieur du conteneur

- Si la valeur est présente (Some / Right), la fonction est appliquée.
- Si elle est absente (None / Left), map ne fait rien : l'état se propage tel quel.
- Le résultat reste du même type de conteneur.



```
val prix: Option[Double] = Some(29.90)

prix.map(p => p * 1.2)
// Some(35.88)

val err: Either[String, Int] = Left("ko")
err.map(_ * 2) // Left("ko") inchangé
```

À retenir : map transforme le contenu sans changer la forme du conteneur (Option reste Option, Either reste Either).

flatMap : enchaîner des étapes

Enchaîne des fonctions qui retournent elles-mêmes un conteneur

- Évite d'obtenir un `Option[Option[A]]` ou un `Either` imbriqué.
- Chaque étape peut échouer indépendamment ; l'échec se propage automatiquement.
- C'est la brique qui permet de construire un vrai pipeline à plusieurs étapes.



```
def parseAge(s: String): Option[Int] =  
  s.toIntOption  
  
def valideAge(a: Int): Option[Int] =  
  if (a > 0) Some(a) else None  
  
parseAge("25").flatMap(valideAge)  
// Some(25)
```

À retenir : `flatMap` enchaîne des fonctions "qui retournent un `Option`/`Either`", là où `map` enchaîne des fonctions "normales".

fold : extraire une valeur finale

Sort du conteneur en fournissant les deux issues

- Sur Option : une valeur par défaut (cas None) et une fonction (cas Some).
- Sur Either : une fonction pour Left et une fonction pour Right.
- Utilisé en fin de pipeline pour produire un résultat exploitable (message, log, statut).



```
val resultat: Either[String, Int] = Right(42)

resultat.fold(
  erreur => s"Échec : $erreur",
  valeur => s"Succès : $valeur"
)
// "Succès : 42"
```

À retenir : fold est le point de sortie du pipeline — on y décide quoi faire du succès et de l'échec.

for : une syntaxe lisible pour le pipeline

Sucre syntaxique au-dessus de map / flatMap

- Rend visible l'enchaînement de plusieurs étapes qui peuvent échouer.
- Fonctionne aussi bien avec Option qu'avec Either.
- Se lit comme une suite d'étapes séquentielles, très proche du raisonnement métier.

```
def parseCommande(l: String)
  : Either[String, Commande] =
  for {
    q <- validerQuantite(quantite)
    p <- validerPrix(prix)
  } yield Commande(id, q, p)
```

À retenir : le for-comprehension est ce que vous utiliserez pour assembler le pipeline complet de l'étude de cas.

Aperçu de l'étude de cas

Contexte

Vous recevez des commandes clients sous forme de lignes de texte brutes (export CSV simplifié) : identifiant, produit, quantité, prix unitaire.



```
CMD001;Clavier;3;29.90  
CMD002;Souris;-1;9.90  
CMD003;;2;15.00
```

Objectif : transformer chaque ligne en une commande valide, ou en un message d'erreur explicite.

Vous devrez construire

- 1 Un parsing sûr des champs numériques avec Option.
- 2 Des règles de validation métier avec Either.
- 3 Une fonction parseCommande combinant les deux via for.
- 4 Un traitement de la liste complète avec map et fold.
- 5 Un récapitulatif final : commandes valides, erreurs, montant total.